ISSN: 3121-4193

CISphere@UoJ

Igniting Intelligent Connections

NEWSLETTER

IEEE Computational Intelligence Society
Student Branch Chapter of the University of Jaffna

Workshops & Tech Talks

Awards

Outreach Activities

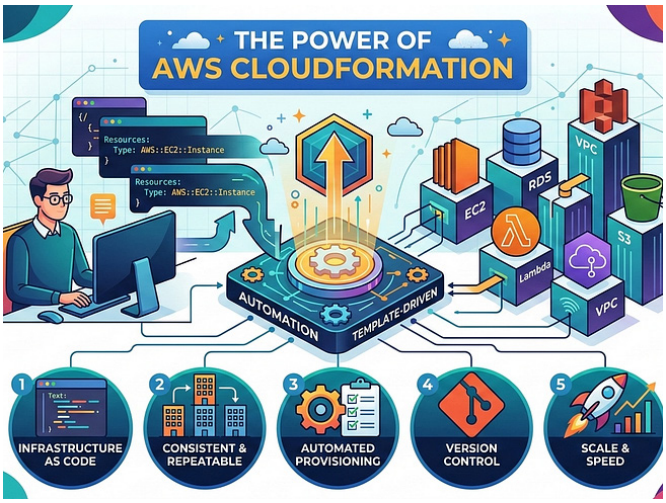
Summer School

IEEE Computational Intelligence Society
University of Jaffna Student Branch Chapter

Volume 01 | 2026

The Power of AWS CloudFormation: Why DevOps Engineers Love It

Mr. Pasidu Chamod
Undergraduate Student
Department of Computer Science
University of Jaffna



Nowadays, most developers are very familiar with the AWS Console. Many of them work daily using the graphical interface. Even DevOps engineers use it. It's interactive, visual, and easy to explore.

But today, I'm going to talk about something very essential for DevOps maturity and automation. A service that changes the way we create infrastructure. That tool is CloudFormation – AWS's built-in Infrastructure as Code (IaC) service.
First question: Why do we even need it? Let's think practically. Our main goal in DevOps is faster delivery, automation, repeatability, less human error, and consistency across environments.

When you create resources manually using the console, it takes time. You might forget a configuration. You might misconfigure security groups. You cannot perfectly replicate the same setup again.

Now imagine you need the same infrastructure for development, testing, staging, and production. Will you manually create everything four times? That's risky and not scalable. This is where CloudFormation becomes powerful.

ElasticBeanstalkSample.template

```
1 {
2  "AWSTemplateFormatVersion" : "2010-09-09",
3
4  "Description" : "AWS CloudFormation Sample Template ElasticBeanstalkSample: Configure and
5
6  "Parameters" : {
7    "KeyName" : {
8      "Description" : "Name of an existing EC2 KeyPair to enable SSH access to the AWS Elast
9      "Type" : "String"
10   }
11 },
12
13 "Resources" : {
14   "sampleApplication" : {
15     "Type" : "AWS::ElasticBeanstalk::Application",
16     "Properties" : {
17       "Description" : "AWS Elastic Beanstalk Sample Application",
18       "ApplicationVersions" : [{
19         "VersionLabel" : "Initial Version",
20         "Description" : "Version 1.0",
21         "SourceBundle" : {
22           "S3Bucket" : { "Fn::Join" : ["-", ["elasticbeanstalk-samples", { "Ref" : "AWS::Re
23           "S3Key" : "elasticbeanstalk-sampleapp.war"
24         }
25       }],
26       "ConfigurationTemplates" : [{
27         "TemplateName" : "DefaultConfiguration",
28         "Description" : "Default Configuration Version 1.0 - with SSH access",
29         "SolutionStackName" : "64bit Amazon Linux running Tomcat 7",
```

So what exactly is CloudFormation? In very simple words, we write a YAML or JSON file that describes all the AWS resources we want. Then we run it, and AWS creates everything automatically according to our instructions. That's it. Instead of clicking buttons in the console, we describe infrastructure as code.
What can we create using it? Almost everything: EC2 instances, S3 buckets, VPC, subnets, security groups, IAM roles, load balancers, and RDS databases – all inside one single file. Now think about that. One file. Entire infrastructure created in minutes.

But is it easy? Let's be honest. At first glance, it feels harder than using the console. When using the console, you click, you select, and you see options clearly. But in CloudFormation, you must know the resource type, you must know the correct properties, you must specify region-based AMI IDs, and you must define dependencies correctly.

For example, if you create an EC2 instance, you must provide the correct AMI ID for that region. You don't "see" it visually; you must define it properly in code. So yes, initially it feels harder, and that feeling is correct.

Then why do we do this "hard" thing? The simple answer is reusability and automation. When you create resources using CloudFormation, everything is grouped into something called a Stack. A Stack is like a packaged infrastructure set.

You can create it, update it, delete it, reuse it later, and version control it in Git. Now imagine you deploy a stack and something fails in the middle. CloudFormation automatically rolls back. It deletes all partially created resources.

That means no half-configured infrastructure, no messy environment, and no manual cleanup. Isn't that powerful?

Another important benefit is consistency. If five DevOps engineers deploy the same CloudFormation template, they get the exact same infrastructure. No difference. No "it works on my account" issues. That is real DevOps quality.

Think like an enterprise. In big companies, infrastructure must be auditable, version controlled, repeatable, and reviewed before deployment. CloudFormation templates can be stored in Git, reviewed via pull requests, approved before deployment, and tracked for changes. You cannot do that with manual console clicks.

So what's the final mindset shift? The console is good for learning and exploration, while CloudFormation is good for production and automation.

In DevOps, we move from manual to automated, from clicking to coding, and from temporary setups to repeatable infrastructure. Yes, at first it feels complex. Yes, writing YAML is not as fun as clicking buttons. But once you understand it, you realize you are not just creating resources – you are designing infrastructure professionally.

So ask yourself: Do you want to create infrastructure once, or do you want to create it infinitely, reliably, and safely? That's the power of CloudFormation.